

Taller Final – Aplicación Web de la Heladería con Django

Objetivo general

El objetivo de este taller es que los estudiantes implementen una aplicación web completa usando Django con sus plantillas para gestionar el inventario, los productos y las ventas de una heladería. Todo debe funcionar en un entorno local usando contenedores Docker.

Enunciado

Una heladería muy popular está buscando digitalizar su menú y parte de su manejo de inventario. Para esto ha decidido contar con nuestra ayuda.

La heladería tiene dos conceptos principales: Productos e Ingredientes.

- A partir de los ingredientes, que pueden ser bases o complementos, es posible armar productos que pueden ser Copas o Malteadas.
- Cada ingrediente tiene nombre, precio, calorías por porción, inventario disponible, si es vegetariano o no, si es sano según regulaciones de salud, y su tipo (base o complemento).
- Las bases tienen un sabor; los complementos pueden reducir su inventario a 0 con una instrucción.

Los productos:

- Están compuestos por 3 ingredientes.
- Tienen nombre, precio al público, tipo (copa o malteada).
- Copas → tipo de vaso.
- Malteadas → volumen en onzas.
- Debe poder calcularse su costo, calorías y rentabilidad.

La heladería quiere saber cuál es el producto más rentable, vender productos revisando inventario, y llevar un contador de ventas del día.

Requisitos mínimos del proyecto

1. Construir la aplicación web completa en Django (sin DRF).
2. Autenticación con el sistema propio de Django (django.contrib.auth).
3. Definir roles de usuario mediante grupos o permisos:
 - Administrador → acceso total.
 - Empleado → CRUD de ingredientes y productos, sin consultar rentabilidad.

- Cliente → solo listar productos, ver calorías y realizar compras.
 - Público (no autenticado) → solo listar productos.
4. Crear un sistema de plantillas atractivo (usar Bootstrap o Tailwind).
5. Implementar vistas para cada caso de uso:
- CRUD de ingredientes y productos.
 - Listado de productos con información de costo, calorías y rentabilidad.
 - Función de venta de productos con actualización de inventario.
 - Login/Logout y control de permisos según rol.

Operaciones obligatorias en la aplicación

Usuarios

- Login / Logout.
- Registro de nuevos usuarios (rol por defecto: cliente).

Productos

- Listar todos los productos.
- Consultar un producto según su ID.
- Consultar un producto según su nombre.
- Ver calorías, costo de producción y rentabilidad.
- Vender un producto (descuenta inventario y registra venta).

Ingredientes

- Listar todos los ingredientes.
- Consultar un ingrediente según su ID o nombre.
- Consultar si un ingrediente es sano.
- Renovar el inventario de un ingrediente (poner a 0 si es complemento).

Guía de implementación en Django

1. Modelos obligatorios

- Ingrediente: nombre, precio, calorías, inventario, es_vegetariano, es_sano, tipo, sabor.
- Producto: nombre, precio_publico, tipo, vaso, volumen_onzas.
- ProductoIngrediente: relación many-to-many entre productos e ingredientes.
- Venta: producto, usuario, cantidad, total, fecha.
- Usuario: usar el modelo de Django (AbstractUser) extendido con campo rol.

2. Vistas y plantillas

- Vistas basadas en clases (CBV) o funciones.
- Plantillas para login, dashboard, listado de productos, detalle, CRUD de ingredientes/productos, etc.
- Estilos con Bootstrap/Tailwind para hacerlo visualmente atractivo.

3. Autenticación y Autorización

- Usar `django.contrib.auth`.
- Definir roles mediante `User.rol` o grupos (`Group`).
- Restringir vistas con decoradores (`@login_required`, `@permission_required`).

4. Docker local

- Configurar un `docker-compose.yml` con Django + PostgreSQL.

Ejemplo mínimo:

```
version: '3.9'
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_USER: heladeria
      POSTGRES_PASSWORD: heladeria
      POSTGRES_DB: heladeria
    ports:
      - '5432:5432'
  web:
    build: .
    command: python manage.py runserver 0.0.0.0:8000
    volumes:
      - ../app
    ports:
      - '8000:8000'
    depends_on:
      - db
```

Entrega esperada

- Proyecto Django completo con modelos, vistas y plantillas funcionando.
- Autenticación y roles implementados.
- CRUD de ingredientes y productos.
- Listado con cálculo de calorías, costo y rentabilidad.
- Función de venta de productos con actualización de inventario.
- Login de usuario.
- Despliegue local en Docker (Django + PostgreSQL).